

Infinite Pokémon: An Agent-Driven Framework for Generative Gameplay

Technical report · Prototype 0.1.0 · Version 0.4 (provisional) · 14 September 2026

Private working draft. This is a versioned, provisional technical report, not a definitive specification. It will evolve with implementation changes and new evidence; claims apply only to the cited source baseline. Source baseline: 2f240d9a46dbdf94f2253d4f7354d38354067a93. Not peer reviewed; no participant study is reported.

Code repository: [Shellishack/infinite-pokemon](#). Repository URL supplied by the project owner. The analysis uses the local source baseline identified above; this report does not assert that the same commit is available on the remote.

Version history: v0.3 documented the implementation review and experiments; v0.4 adds explicit provisional status, repository attribution, and preserved versioned deliverables. It introduces no new gameplay changes or experimental results.

The adventure never ends.

Abstract

Infinite Pokémon is a self-hosted, two-dimensional creature-collecting prototype that extends its world through structured proposals from a host-connected AI harness. An authoritative engine controls movement, collision, combat, inventory, rewards, and persistence. The harness supplies descriptions, narrative hooks, terrain features, content profiles, NPC policies, and constrained interiors. Proposals are compiled and checked against saved-world dependencies. Checkpoints support divergent runs; generation inputs come from each run's database rather than a global transcript. A distance-based scheduler prepares nearby maps with configurable depth and concurrency. This report reviews the implementation, skills, multiplayer, and presentation. Fresh verification includes a sequential 99-test pass and successful desktop and generated-scene checks, alongside an initial failed/cancelled regression run and desktop timeout retained as evidence. These checks exercise software behavior, not narrative coherence, enjoyment, unlimited novelty, or real-time inference. Bounded context, template geometry, incomplete services, persistence coupling, and publication bookkeeping remain limitations. Proposed evaluation separates operational correctness from generative quality and player experience.

1. Purpose and review scope

The intended experience is a handheld-style adventure with persistent places and an expanding frontier. Players should revisit towns, retain consequences, meet recognizable NPCs, and travel with friends. Content should arrive before it is needed where possible, without allowing a model to redefine mechanics or invalidate a saved world.

Here, generative gameplay means gameplay whose available content is extended during play through generation. Agent-driven describes a bounded task loop: receive observations and a skill, perform a content task through the harness, and return structured output for acceptance. It does not mean every NPC runs a reasoning process. Locomotion remains engine code.

“Infinite” means no prescribed final authored block. Storage, time, numerical representation, schemas, provider allowance, and mechanics remain finite. The tagline is an aspiration, not a scientific claim about engagement or coherence.

This revision reviews the code paths supporting the report's claims and inventories 129 tracked source, configuration, test, skill, and documentation files with hashes. That inventory provides provenance, not formal verification or an exhaustive security audit. The older draft is archived. This report supersedes earlier proposals as the description of current behavior; it does not assert that every planned feature exists.

2. Related systems

Summerville et al. survey procedural content generation through machine learning, including functional content and control challenges [1]. This prototype does not train a model: it combines an existing language-model harness with a deterministic compiler. [PCGML survey](#).

MarioGPT generates text-conditioned tile levels and explores diversity with novelty search [2]. Infinite Pokémon instead compiles descriptions and bounded recipes into protected layouts. Its present focus is persistence and admission around generation, not a new learned level generator. [MarioGPT](#).

Generative Agents stores experiences, synthesizes reflections, and retrieves information for planning [3]. Our NPC loop is narrower: limited memories, revisions, dialogue, three intention labels, and movement policies. It does not reproduce that reflection architecture or evaluation. [Generative Agents](#).

Voyager combines an automatic curriculum, executable skills, and feedback for an embodied Minecraft agent [4]. Our skills are maintained instructions for content workers; players and test drivers do not learn action programs. [Voyager](#).

CALYPSO explores LLM assistance for Dungeon Masters [5]. This prototype automates acceptance for a narrower output space. FIREBALL associates role-playing dialogue with structured game state [6]; it motivates grounding but does not establish the sufficiency of this implementation's context. No comparative evaluation has been performed. [CALYPSO](#), [FIREBALL](#).

3. Architecture and organization

TypeScript, React, Phaser, Vite, Node.js, SQLite, WebSockets, and Electron form the stack. Electron entry/preload files remain CommonJS JavaScript; all utilities in scripts/ are TypeScript. React manages menus, Phaser renders the overworld, and Node owns canonical state. Networking uses ws directly, without Colyseus, Redis, distributed workers, or central matchmaking.

Location	Responsibility
desktop/	Window, trusted preload, shell, server supervision, icons and fullscreen
game/client/	HTML, React UI, Phaser, prediction, battle presentation and audio
game/engine/	Authoritative actions, maps, economy, interiors and NPC runtime
game/server/	HTTP/WebSocket, harness, queue, snapshots, storage, saves and reset
game/shared/	Types, schemas, collision and communication contracts
game/assets/	Bundled artwork, branding, music and effects
game/content/	Authored five-map tutorial pack
scripts/ and skills/	Offline generators and versioned instructions

Vite starts from game/client/index.html, serves game/assets/, and emits dist/. The host compiles to dist-server/game/server/index.js. Electron launches it from the repository working directory. Startup requires a Node executable and built output; this is not a self-contained installer.

Engine/server separation is organizational, not complete persistence decoupling. Engine modules accept Store, some use SQLite directly, and startup migrations remain in engine code. Browser modules consume shared definitions; authoritative gameplay runs on the host even in single-player.

4. Connection and multiplayer

The title offers Single player and Multiplayer, with Host and Join under multiplayer. Hosts connect local Codex and accept token use; guests join by address without provider authentication. There is no visible game account, but technical identity exists: opaque trainer tokens, hashed storage, and browser-local identity. Reusing a nickname does not recover cleared browser storage.

Verification sends a fresh nonce and expects JSON with the same nonce and status ok. It omits generation skills and instructs the harness not to use tools. The default deadline is 30 seconds. Remembered consent permits automatic confirmation after restart; live verification is reused. A short prompt does not guarantee low latency or total tokens because session overhead remains external. This revision makes no fresh provider-latency measurement.

Hosts can disconnect, switch account, or select another compatible Codex executable. Disconnect invalidates generation and cancels work while preserving completed maps. There is no generic provider plug-in implementation. The normal UI has no job-budget control, although legacy server budget handling remains.

Guest and loopback administration listeners are separate, normally ports 8787 and 8788; Electron requests available ports. Host operations require a per-process local token. Single-player blocks public gameplay admission. WebSockets authenticate in their first message, retain one socket per trainer, and cap online trainers at eight and stored profiles at 64. These are configured limits, not measured WAN capacity.

The server checks schemas and ownership, limits command frequency, and broadcasts every 100 ms. Other players receive scene-local position/appearance summaries rather than full inventories; conversations and NPC memory receive filtering. Leaderboard statistics are intentionally shared. This is a local-host trust model, not an audited multi-tenant service. Docker/Caddy are optional; a reverse proxy alone does not solve NAT traversal.

5. State, context, and admission

5.1 Authority

SQLite stores players, maps, events, receipts, NPC memory/motion, definitions, explored cells, jobs, and metadata. World.apply validates actions and stores results with command receipts in a transaction. Duplicate commands return prior results rather than duplicate rewards or item use. The client does not determine damage, capture, currency, or progression.

Generated output is data, not executable code. Region-story schema version 3 permits descriptions, greetings, hooks, four biome labels, bounded terrain/content recipes, up to twelve NPC bindings, and up to two interior designs. Arbitrary generated JavaScript is not evaluated.

5.2 Context selection

Store.snapshot writes a staging directory and renames it into a final snapshot. A manifest identifies lineage, dependencies, and hashes. Compact context contains destination/nearby-map summaries, selected player state, world choice, terrain edges, and events. Archive files contain additional views; context/current.json points to the latest snapshot.

Maps are selected within Manhattan distance two of the target, excluding unfinished fallback drafts except that target. Of the latest 100 events, compact context filters relevant entries and keeps at most 24. Inventory and visited-place summaries are bounded. The prompt is therefore not the entire gameplay history, despite current

harness wording. Old facts can disappear unless retained in state; semantic retrieval and durable narrative summaries are absent.

Hashes support provenance and dependency comparisons. Files are neither signed nor write-protected; hashes do not prove the model read an input or prevent host tampering. The database remains authoritative and files are materialized inputs.

5.3 Acceptance

A map job records its snapshot and reserves an attempt before dispatch. It requests a schema-conforming story and snapshot identity. Before publication it checks the active verified connection, epoch, run identity, unchanged/unpublished destination, world choice, and dependency hashes/publication states. Geometry is compiled and validated.

Accept = schema-valid AND snapshot-match AND current-run/epoch AND target-unchanged/unpublished AND dependencies-unchanged AND geometry-valid.

This summarizes code checks, not narrative serializability. Event sequence is recorded but is not a blanket fence: events may occur during generation. Facts outside dependencies may be stale or absent. Valid JSON can contradict dialogue; there is no general contradiction detector.

Prepared and published differ. A background map can be prepared before visitation. Ordinary generation refuses to overwrite published maps, while backed-up source-version migrations are a separate exception. Persistence is not absolute immutability across upgrades.

5.4 Artifacts and failure

New diagnostics and compiled maps go under each run's generated-content/ folder. Git and Docker ignore it; it is not a public asset folder. NPC proposals are also logged there; observations live in the run's harness folder. Legacy generation/ logs are not migrated. Checkpoints preserve databases, not all diagnostic files.

Publication and the subsequent diagnostic write share an outer try/catch. A file-write failure after database commit can label a job failed although the map exists. No transactional outbox couples SQLite and JSON. This is a review finding requiring separation of publication and diagnostic outcomes.

6. Prefetch geometry and concurrency

Depth controls preparation distance, not zoom. Four-direction adjacency forms a Manhattan diamond. Ring k contains $4k$ blocks, so surrounding count $N(d) = 2d(d + 1)$; including center gives $1 + 2d(d + 1)$.

Depth	Surrounding	Including center
0	0	1
1, default	4	5
2	12	13
3, maximum	24	25

These are analytical counts, not throughput measurements. One-block movement introduces at most $2d + 1$ new coordinates into a prepared diamond: three, five, or seven at depths one, two, or three. Cached places reduce work. Multiplayer uses the union of neighborhoods; separated players can exceed 25 total blocks.

Queue ordering uses minimum distance to active origins, with demanded travel breaking equal-distance ties. Missing active players fall back to host/first player or spawn. Prepared maps are reused and jobs deduplicated.

Decreasing depth rebuilds queued demand without necessarily stopping running work. Maps accumulate without eviction.

Batch size means concurrency, default three and configurable one to eight. Each map has a separate task/output, not a multi-map prompt. Slots refill when jobs finish or settings change; reducing concurrency lets running work finish. NPC jobs run serially after map work drains, so frontier demand can delay them. Job records and world-state reconstruction replace a durable distributed broker.

Unfinished destinations hold travel at the origin and display queued/generating/failed/paused/disconnected/budget states, elapsed time, and queue position where available. These are not percentages. Readiness triggers traversal only if the player remains online at the expected origin. Cancel prevents late warps; failures require retry.

The bottom-left indicator shows active/queued maps. The agent panel exposes model, effort, reported tokens, tool lifecycle labels, and validation/failure events, not reasoning or raw output. Ideally, M independent equal-duration jobs at concurrency b require $\text{ceil}(M/b)$ service intervals. Real setup, limits, retries, and dependencies prevent this from being a latency guarantee.

7. Checkpoints and reset

A run is a writable database with identity/ancestry. Progress persists during play. A checkpoint is a VACUUM INTO SQLite snapshot, renamed and cataloged with hash and parent. Default runs and initial checkpoints are created once meaningful player state exists. Saving does not itself fork; branching is explicit.

Forking verifies the checkpoint hash, copies its database, creates run/world identities, preserves inherited facts, rotates trainer tokens, clears jobs/receipts and consent, and defaults to single-player. Future context comes from that copy, excluding later parent/sibling events. This is ancestry, not branch merging.

Unfinished shared battles block checkpoints; online guests constrain switching. Storage isolation does not prove semantic consistency. Filesystem snapshots and catalog writes are not one transaction; failures can leave orphan artifacts.

Full reset closes children, cancels generation, stages owned paths, recreates state, and removes staged data after success. Paths and symbolic links are checked. Both new and legacy generation folders are covered; unrelated files and external CLI sign-in are preserved. Tests exercise reset behavior, not every crash/filesystem failure.

8. Terrain, NPCs, interiors, and skills

8.1 Terrain

Blocks are 32 by 24 tiles of 16 pixels. Opening layouts are town, north forest, south river, east coast, and west ruins. Preview has five authored outdoors and six rooms. Full play defaults to five allocated blocks, but neighbors may still generate; preview supplies the fully prepared start.

The compiler preserves paths, exits, and interaction access while adding bounded trees, ponds, flowers, grass, and stones. Validation checks tile values, bounds, identities, exits, tutorial roles, practice grass, healing/exit access, and door arrivals. Edge context records surface mixtures and run-length-encoded passability.

Edge observations are not a seam solver. Gradual city-to-field transitions are guidance; continuous blending and arbitrary geometry are absent. The code's Region means one block, not a group of dozens of blocks.

8.2 NPC behavior

Policies choose stationary, wander, or patrol: radius one to three tiles, intervals two to eight seconds, and at most six relative waypoints. The compiler validates actor identity, bounds, waypoints, and protected services. Healers, merchants, and nursery keepers stay stationary.

NpcRuntime persists poses separately by block/scene/actor and source hash. It updates occupied scenes, freezes near players or battles, reserves motion cells, avoids doors/spawn, and rejects access-blocking moves. Motion has a 320 ms trajectory. Wander order is deterministic from actor identity and step count, not model calls per step.

Live planning addresses the outdoor guide: greet/guard/rest, intention, dialogue, memory, and optional policy. Revision, region hash, run, epoch, and choice fence results. Memory retains twelve strings. Other NPCs can receive initial map policies, but are not independent live reasoning agents. This is bounded characterization, not a simulated society.

8.3 Interiors and services

Proposals rename prepared rooms and arrange tables, counters, bookcases, beds, and rugs with bounded footprints/counts. Shell, doors, spawn, terminal, and service actors remain protected. Overlap and access checks precede acceptance; layouts affect movement and inspection.

Healing, shops, nursery, and save-menu functionality exist. References describe fuller community centers, trades, geographic regions, and Dojo masters, but capabilities explicitly mark trading, regional registry, Dojos, custom buildings, image generation, and asset import unsupported. Naming a room Dojo grants no master or progression system.



Figure 1. Current browser capture of a hand-authored structured proposal compiled into an interior. Tests check furniture interaction, healing, and exit. This is fixture rendering, not a live model-generated room.

8.4 Skills and trust

The root skill grants contextual creative freedom and links runtime, continuity, terrain, building, regional, visual-asset, and economy references. Separate NPC/interior skills define contracts. Documents guide choices; schemas and runtime enforce mechanics. Broad catalogs must be interpreted through narrower capabilities.

Codex app-server transport uses stdio, ephemeral threads, read-only sandbox requests, no approvals, and disabled web search. Interactive requests are declined. Instructions restrict reads to supplied context and linked references,

treating game text as untrusted. Read-only execution is not a per-file read allowlist, and instructions are not proof against injection. No adversarial evaluation is claimed.

9. Variety and presentation

Six base species and twelve additional authored profiles are included. Generated profiles use bounded elements, traits, forms, colors, patterns, and base mechanics. Stable IDs derive from selected normalized fields; definitions are stored once. This expands a finite recipe vocabulary without proving infinite mechanical novelty. Some traits affect HP, damage, or capture; curious, swift, and radiant are flavor traits.

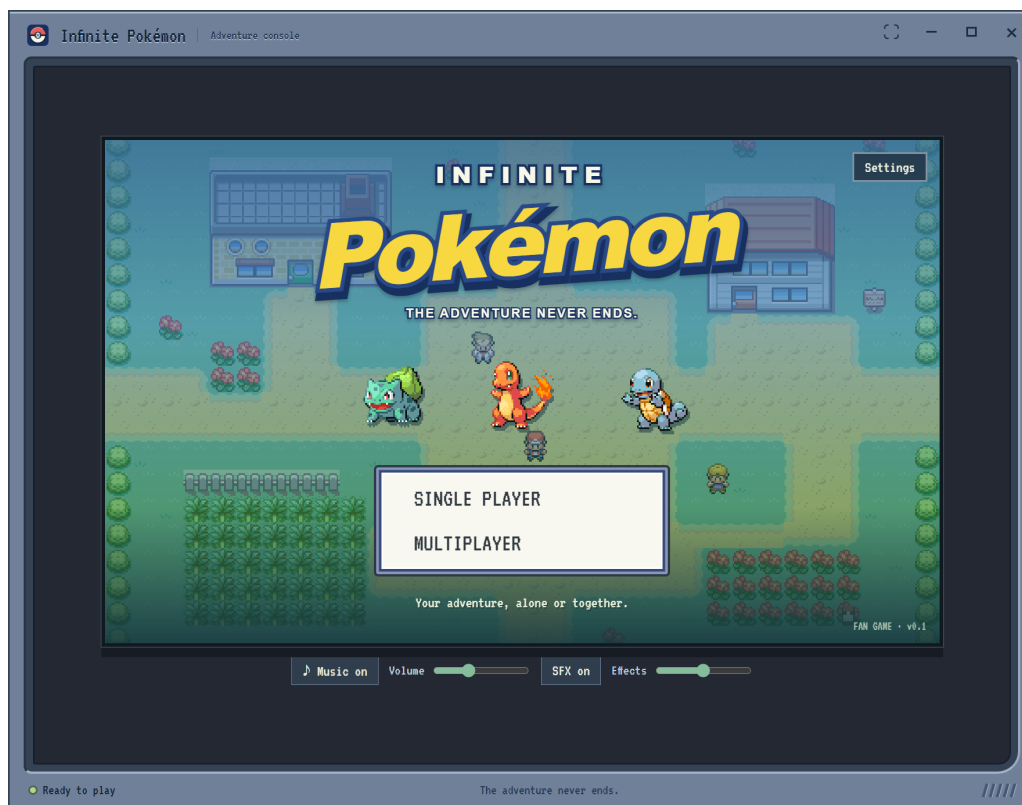


Figure 2. Title screen from the successful follow-up desktop check. It demonstrates the interface, not usability metrics or live generation.

Hybrid breeding is deterministic, not inference. Two healthy owned companions at level five or higher can produce an egg for 120 coins, subject to cooldown and one-egg limits. Identity derives from sorted parent species; traits/art combine by fixed rules. Hatching needs 32 steps; parents rest for 64. Repeated species pairs reuse the profile. This is not biological genetics or learned evolution.

Shops offer eight authored items plus bounded generated goods. Code controls prices, effect tiers, proximity, funds, and quantities. Bikes and vehicle profiles change outdoor speed but preserve collision; arbitrary flight is absent. Run-local leaderboards count captures, distinct species, blocks, explored cells, and trainers defeated. Starters/hatchlings are not captures. No global ranking exists.

Held movement predicts one tile with acknowledgements, interpolation, and correction. Walking takes 160 ms; bikes 110 ms, with a faster tier of 90 ms. Latency still affects responsiveness. Hitboxes, explicit doors, and facing-tile inspection provide discrete mechanics rather than physics.

The first tutorial begins at spawn; later lessons use ordered personal flags and a lesson location rather than compass direction. Intro, starter, dialogue, encounters, and battles have explicit UI states. Escape controls the menu; dialogue offers Continue and keyboard dismissal. Encounter shutters/reveal respect reduced motion.

Fight/Bag/Party/Run, sequential messages, HP animation, and SFX present server-owned outcomes. The battle system is simplified, not a full original-game reproduction.

Electron provides a custom frame, icons, startup/retry, viewport fitting, and F11 fullscreen. A trusted shell has narrow native access; the game has no direct Node bridge. Profile-directory compatibility preserves saves across branding changes.

Twelve original music cues and twenty-two effects are synthesized offline by TypeScript into PCM WAV. No runtime audio-generation provider exists. Playback unlocks on gestures, caches, crossfades, pauses when hidden, and has separate music/SFX settings. Effects follow accepted actions or battle playback, with duplicate suppression and bounded voices. Reference artwork credits in `game/assets/classic/` are distinct from original audio and procedural recipes.

10. Verification evidence

New checks use isolated directories and deterministic fixtures/protocol stubs, with no live Codex jobs. Screenshots are actual captures. Source hashes and copies are under revisions/2026-09-14/. Earlier outputs are not relabeled as current measurements.

Check	Observed result	Interpretation
Initial default unit run	99 reported: 86 pass, 1 fail, 12 cancelled	Snapshot rename error plus pending-promise cancellations; not clean
Sequential follow-up	99 pass, zero failures/cancellations	File concurrency one; does not erase instability
Initial desktop smoke	Single player locator timeout	Cause not established
Desktop follow-up	Passed	Startup, frame, fullscreen, preview, menu, fit, shutdown
Generated scenes browser	Passed	Fixture movement, interior, inspection, healing, exit

Commands are `npx tsx --test --test-concurrency=1 tests/*.test.ts`, `npm run test:desktop`, and `npx tsx tests/generative-scenes-e2e.ts`. Sequential TAP reports 27,281.0188 ms, a suite duration rather than gameplay/model latency. Desktop reports Node 22.21.1, Electron 39.8.10, and Chromium 142.0.7444.265. The copied lockfile preserves resolved dependencies.

The initial startup test reported an unhandled EPERM during snapshot-directory rename; harness tests also reported pending promises after the event loop resolved. The initial unit run overlapped desktop automation, and follow-up also changed file concurrency. These runs do not isolate the failure cause. No success rate, confidence interval, or capacity estimate is inferred. A stable evaluation harness is itself outstanding work.

Unit coverage includes receipts, combat/PP, economy, geometry, NPC policies, forks, stale proposals, preview, network/session behavior, reset, audio, and streaming. Browser checks are scenarios, not exhaustive state-space exploration. This revision does not rerun every historical browser suite or validate WAN play, packaging, assistive technology, or long autonomous sessions.

An archived live trial dated 9 September records one verification and five map attempts with two accepted maps. It used older deadlines/context and omitted failed-turn usage. This is historical context, not an estimate of current cost, quality, latency, or acceptance. Old benchmarks are not promoted to current results.

11. Code-review findings

- Snapshot setup can escape failure handling. The observed EPERM occurred before the guarded generation body, producing an unhandled rejection; protect setup as part of the job lifecycle.
- Context is selective. Complete-history prompt wording exceeds its bounded projection. Durable summaries and retrieval require implementation and evaluation.
- Validity is not truth. Schema/hash checks do not establish narrative consistency, originality, balance, or engaging quests.
- Geometry is constrained. Templates support variation; continuous blending, arbitrary buildings, image import, trading, and regional Dojos remain future work.
- Publication and diagnostics can disagree. Post-commit JSON failure can misreport a persisted map as failed. Separate those outcomes.
- Runtime is locally coupled. SQLite, checkpoint copies, per-view queries, and NPC access searches are synchronous. No retention policy or large-world load study establishes scaling.
- Scheduling has costs. Separated players increase demand, NPCs wait behind maps, and phases are not completion forecasts.
- Tests are not uniformly stable. Pending promises and a UI timeout occurred before successful follow-ups. Earlier green runs do not prove deterministic reliability.
- Trust is bounded. Local authentication and read-only requests help, but no formal sandbox audit or injection benchmark exists.
- Scientific claims are prospective. No participant sample, engagement study, baseline comparison, or demonstrated endless novelty supports stronger conclusions.

The companion `CODE_REVIEW.md` links findings to exact source locations. No gameplay source was modified to improve the report's test outcomes.

12. Proposed evaluation

First stabilize fixtures and retain failures, seeds, and dependency versions. Compare bounded context with summary/retrieval variants over scripted branching histories and planted facts. Measure omissions and contradictions against annotated history, with sibling-branch facts as negative controls. Distinguish inventions from intended story changes.

For streaming, sweep depth zero through three and concurrency one, three, and eight over fixed traces, including reversals and separated players. Record service/queue/destination wait, retries, accepted/unused maps, storage, and tokens. Separate cold/warm caches, log model/effort/deadlines/context size, and account for aborted work. Simulated delay checks the scheduler but cannot substitute for provider-cost measurements.

For NPCs/interiors, compare stationary/policy-driven actors and authored/generated layouts. Measure blocked interactions, failed movement, access, behavior legibility, and dialogue-state agreement. Validate mechanics before studying perceived variety.

After reliability improves, a consented study could compare authored-only and generated-content play. Predefine duration, counterbalancing, sample-size justification, stopping rules, and coherence, agency, repetition, and enjoyment measures. Ratings and operational traces should remain separate. None of these experiments has been performed for this report.

13. Conclusion

Infinite Pokémon implements a practical separation between generative work and authoritative gameplay. It provides a prototype for examining how saved histories, compiled proposals, bounded behavior, and asynchronous preparation coexist in a persistent adventure. It does not establish unlimited coherent content or sustained engagement. Reproducible evaluation, stronger context management, clearer publication bookkeeping, and stable tests are the next steps.

References

- [1] Adam Summerville et al. *Procedural Content Generation via Machine Learning (PCGML)*. Revised 2018. [arXiv:1702.00539](#).
- [2] Shyam Sudhakaran et al. *MarioGPT: Open-Ended Text2Level Generation through Large Language Models*. 2023. [arXiv:2302.05981](#).
- [3] Joon Sung Park et al. *Generative Agents: Interactive Simulacra of Human Behavior*. 2023. [arXiv:2304.03442](#).
- [4] Guanzhi Wang et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. 2023. [arXiv:2305.16291](#).
- [5] Andrew Zhu et al. *CALYPSO: LLMs as Dungeon Master's Assistants*. AIIDE, 2023. [Primary publication](#).
- [6] Andrew Zhu et al. *FIREBALL: A Dataset of Dungeons and Dragons Actual-Play with Structured Game State Information*. ACL, 2023. [Primary publication](#).

Primary related-work landing pages were checked on 14 September 2026. They provide context, not independent validation of this prototype. Implementation claims derive from the source and evidence inventory.